

Sisteme de Operare

Ghid de pregătire: Pipe-uri, FIFO și Thread-uri
(mutex, read-write lock, condition variable, barrier, semaphore)

Cuprins

1	Introducere și scop	3
2	Pipe-uri anonime (pipe)	4
2.1	Ce este un pipe	4
2.2	Crearea unui pipe	4
2.3	Exemplu minimal: comunicare părinte-copil	4
2.4	Regulile de funcționare ale unui pipe (de memorat)	5
2.5	Folosirea unui pipe ca semafor între procese	5
3	FIFO (pipe-uri cu nume / named pipes)	6
3.1	Diferența față de pipe-ul anonim	6
3.2	Crearea unui FIFO	6
3.3	Exemplu minimal: scriitor și cititor (procese separate)	6
3.4	Regulile de open pe un FIFO (de memorat exact)	6
3.5	Câte FIFO-uri poate deschide un proces? — tip de întrebare	7
3.6	De ce nu este indicată comunicarea bidirecțională printr-un singur FIFO	7
4	Thread-uri: creare și sincronizare	8
4.1	De ce avem nevoie de thread-uri (recapitulare scurtă)	8
4.2	Creare și sincronizare de bază: <code>pthread_create</code> / <code>pthread_join</code>	8
4.3	Secțiune critică: exemplul clasic de race condition	8
5	Mutex (exclusivitate reciprocă)	9
5.1	Ce este un mutex	9
5.2	API de bază	9
5.3	Exemplu: protejarea unei secțiuni critice cu mai multe thread-uri	9
5.4	Cum prevenim deadlock-urile cauzate de mutex-uri	9
6	Semafoare POSIX (<code>sem_t</code>)	11
6.1	Ce este un semafor	11
6.2	API de bază	11
6.3	Exemplu rezolvat: limitarea concurenței (cel mult K thread-uri simultan)	11
6.4	Producător-consumator cu semafoare (buffer de capacitate N)	12
6.5	Stare unui thread la <code>sem_wait</code>	13
7	Read-Write Lock (<code>pthread_rwlock_t</code>)	14
7.1	Diferența față de mutex	14
7.2	Efectul înlocuirii mutex-urilor cu <code>rwlock</code> -uri	14

8	Variabile de condiție (pthread_cond_t)	15
8.1	De ce avem nevoie de ele	15
8.2	API de bază	15
8.3	Ce face <code>pthread_cond_wait</code> cu mutex-ul primit ca argument	15
8.4	De ce se verifică condiția într-o buclă <code>while</code> după <code>cond_wait</code>	15
8.5	Exemplu: producer-consumer cu variabilă de condiție (alternativă la semafoare) . . .	16
9	Bariere (pthread_barrier_t)	17
9.1	Semantica unei bariere	17
9.2	Regulă pentru exerciții de tip "găsiți valori pentru $T, N_1, N_2, \dots$ "	17
9.3	Efectul unei bariere inițializate cu $N = 1$	17
10	Tabel comparativ rapid	18
11	Analiza tipurilor de subiecte de examen	19
11.1	Categorii pentru Pipe / FIFO	19
11.2	Categorii pentru Thread-uri	19
11.3	Recomandare de strategie generală pentru examen	20

1 Introducere și scop

Acest document este un ghid de pregătire axat strict pe două teme care apar frecvent la examenul de Sisteme de Operare:

- **Pipe-uri și FIFO** (comunicare între procese prin canale de tip pipe);
- **Thread-uri** (creare/join, mutex, read-write lock, variabilă de condiție, barieră, semafor).

Temele `exec*/popen/dup/dup2` și `signal/kill` sunt **excluse intenționat** din acest ghid (urmează a fi tratate separat), exact ca cerere. Acolo unde exercițiile din arhiva de subiecte amestecă `exec` cu `pipe`, vom trata doar partea de pipe.

Structura fiecărei secțiuni este aceeași:

1. un exemplu de cod **cât mai simplu** care introduce conceptul;
2. **regulile** exacte de funcționare (ce trebuie memorat ad litteram pentru examen);
3. o variantă de cod care **rezolvă o problemă tipică** de examen folosind conceptul;
4. observații / capcane frecvente.

La final, secțiunea 11 face o trecere sistematică prin **tipurile** de întrebări întâlnite în arhiva de subiecte (pe FIFO/pipe și pe thread-uri), grupate pe categorii, astfel încât să recunoști imediat ce ți se cere la un subiect nou.

2 Pipe-uri anonime (pipe)

2.1 Ce este un pipe

Un **pipe** este un canal de comunicare unidirecțional între procese **înrudite** (de regulă părinte–copil, create prin `fork`), care funcționează ca o coadă FIFO (First-In-First-Out) de octeți, gestionată complet de kernel, fără nume în sistemul de fișiere.

Un pipe are **două capete**, reprezentate ca doi descriptori de fișier:

- `fd[0]` — capătul de **citire**;
- `fd[1]` — capătul de **scriere**.

2.2 Crearea unui pipe

```
1 #include <unistd.h>
2
3 int fd[2];
4 int rc = pipe(fd);
5 // rc == 0 -> succes
6 // rc == -1 -> eroare (vezi errno)
7 // fd[0] = capat citire, fd[1] = capat scriere
```

2.3 Exemplu minimal: comunicare părinte–copil

```
1 #include <unistd.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int main() {
6     int fd[2];
7     pipe(fd);
8
9     if (fork() == 0) {
10         // PROCES COPIL: scrie in pipe
11         close(fd[0]); // nu are nevoie de capatul de citire
12         char *msg = "salut din copil";
13         write(fd[1], msg, strlen(msg) + 1);
14         close(fd[1]);
15         return 0;
16     } else {
17         // PROCES PARINTE: citește din pipe
18         close(fd[1]); // nu are nevoie de capatul de scriere
19         char buf[100];
20         int n = read(fd[0], buf, sizeof(buf));
21         write(1, buf, n); // afiseaza mesajul
22         close(fd[0]);
23         wait(NULL);
24     }
25     return 0;
26 }
```

Observație esențială: descriptorii unui pipe sunt *duplicați* la `fork()`. De aceea, fiecare proces trebuie să închidă capătul de care *nu* are nevoie — altfel kernelul "vede" în continuare un capăt de scriere deschis (la copil sau la părinte), iar comportamentul de blocare (vezi mai jos) nu se mai declanșează corect.

2.4 Regulile de funcționare ale unui pipe (de memorat)

Citirea (**read**) dintr-un pipe.itemize

Dacă există date în buffer, **read** returnează *imediat* cât are disponibil (poate fi mai puțin decât s-a cerut — citire parțială permisă).

Dacă pipe-ul este **gol** și **există cel puțin un capăt de scriere deschis** undeva (la orice proces), **read** se **blochează** până apar date sau până se închid toate capetele de scriere.

Dacă pipe-ul este gol și **toate** capetele de scriere au fost închise, **read** returnează **0** (semnifică EOF, nu eroare).

Scrierea (**write**) într-un pipe.itemize

Dacă există loc suficient în buffer-ul intern al pipe-ului, **write** reușește imediat.

Pe Linux, scrierile de cel mult PIPE_BUF octeți (de obicei 4096) sunt **atomice**: nu se interclasează cu scrieri din alte procese.

Capacitatea unui pipe este **limitată** (de obicei 64KB pe Linux, configurabil). Dacă buffer-ul este plin, **write** se **blochează** până se mai eliberează spațiu (adică până cineva mai citește).

Dacă **toate** capetele de citire au fost închise, iar un proces încearcă să scrie, procesul respectiv primește semnalul SIGPIPE (care, dacă nu e tratat, termină procesul); apelul **write** returnează -1 cu **errno** = EPIPE dacă semnalul este ignorat/tratat.

Notă de examen: Atenție la formularea capcană: "pipe plin al cărui capăt de scriere a fost închis de **toate celelalte procese**". Aici procesul care scrie încă deține propriul capăt de scriere deschis (altfel n-ar putea apela **write**); faptul că *alte* procese și-au închis copiile lor ale capătului de scriere **nu schimbă nimic** — comportamentul depinde doar de (a) dacă mai există vreun capăt de citire deschis și (b) dacă buffer-ul e plin. Atâta timp cât există un cititor, **write** pe un pipe plin pur și simplu se **blochează** normal, așteptând ca cititorul să consume date și să facă loc; nu se declanșează SIGPIPE (acesta apare doar când *toate* capetele de citire — nu de scriere — sunt închise).

2.5 Folosirea unui pipe ca semafor între procese

Un pipe poate sincroniza două procese fără semafoare POSIX: ideea este că un **read** pe un pipe gol blochează procesul, exact ca un P() (down) pe un semafor, iar un **write** de un octet "eliberează" procesul care așteaptă, ca un V() (up).

```
1 int fd[2];
2 pipe(fd);
3
4 // Proces A: trebuie sa "semnaleze" procesul B
5 write(fd[1], "x", 1); // echivalent V() / sem_post
6
7 // Proces B: trebuie sa "astepte" semnalul de la A
8 char c;
9 read(fd[0], &c, 1); // echivalent P() / sem_wait (blocant pana scrie A
  )
```

Pentru un semafor cu valoare inițială *N*, se scriu *N* octeți în pipe înainte de start; fiecare **read**(1 octet) consumă un "permis", iar fiecare **write**(1 octet) adaugă un permis înapoi.

3 FIFO (pipe-uri cu nume / named pipes)

3.1 Diferența față de pipe-ul anonim

Un **FIFO** este, conceptual, același mecanism de coadă de octeți gestionat de kernel, dar are **un nume în sistemul de fișiere** (vizibil cu `ls`, tip de fișier `p`). Avantajul major: **procesele neînrudite** (care nu provin dintr-un `fork` comun) pot comunica prin el, atâta timp cât cunosc calea către fișierul FIFO.

3.2 Crearea unui FIFO

```
1 #include <sys/types.h>
2 #include <sys/stat.h>
3
4 mkfifo("/tmp/canal", 0666); // creeaza fisierul FIFO pe disc (doar
    metadata, fara date)
```

Echivalentul din linia de comandă: `mkfifo /tmp/canal`.

3.3 Exemplu minimal: scriitor și cititor (processe separate)

writer.c

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <string.h>
4
5 int main() {
6     int fd = open("/tmp/canal", O_WRONLY); // se blocheaza pana se
        deschide un cititor
7     char *msg = "mesaj prin FIFO";
8     write(fd, msg, strlen(msg) + 1);
9     close(fd);
10    return 0;
11 }
```

reader.c

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main() {
6     int fd = open("/tmp/canal", O_RDONLY); // se blocheaza pana se
        deschide un scriitor
7     char buf[100];
8     int n = read(fd, buf, sizeof(buf));
9     write(1, buf, n);
10    close(fd);
11    return 0;
12 }
```

3.4 Regulile de open pe un FIFO (de memorat exact)

Comportamentul implicit (fără `O_NONBLOCK`) este sincron — "rendez-vous": itemize

`open(fifo, O_RDONLY)` se **blochează** până când **cel puțin un alt proces** deschide același FIFO pentru scriere.

`open(fifo, O_WRONLY)` se **blochează** până când **cel puțin un alt proces** deschide același FIFO pentru citire.

Practic, cele două apeluri de `open` se "așteaptă" reciproc — niciunul nu returnează până nu există o pereche cititor+scriitor.

Cu `O_NONBLOCK`, comportamentul se schimbă: itemize

`open(fifo, O_RDONLY | O_NONBLOCK)` **reușește imediat**, indiferent dacă există vreun scriitor (ulterior, `read` se va comporta normal: blochează/EOF după regulile pipe-ului).

`open(fifo, O_WRONLY | O_NONBLOCK)` **eșuează** cu `errno = ENXIO` dacă nu există încă niciun proces care a deschis FIFO-ul pentru citire.

`O_RDWR` pe un FIFO are comportament nedefinit/discutabil din punct de vedere POSIX (pe Linux nu blochează, dar se evită în practică — un proces nu ar trebui să se asculte/comunice cu el însuși pe un singur FIFO).

Notă de examen: Întrebare tip: "Un proces deschide un FIFO pentru scriere și apoi imediat scrie 100 octeți; dacă niciun proces nu l-a deschis pentru citire, ce se întâmplă cu `write`?" Capcana: **dacă `open` este fără `O_NONBLOCK`**, atunci procesul **nu ajunge niciodată la `write`** fără ca un cititor să existe deja — pentru că `open` în sine s-ar fi blocat până apăsarea un cititor. Deci, în regim implicit (blocant), enunțul "a deschis cu succes și apoi scrie fără cititor" presupune deja că un cititor a existat la un moment dat (cel puțin cât să "destupe" `open`); dacă acel cititor s-a închis chiar înainte de `write`, iar el era **singurul** cititor, scrierea va declanșa SIGPIPE/EPIPE (capătul de citire e complet închis). Dacă, în schimb, `open` folosește `O_NONBLOCK`, apelul ar fi eșuat cu `ENXIO` înainte de a ajunge la `write`.

3.5 Câte FIFO-uri poate deschide un proces? — tip de întrebare

Acest tip de întrebare testează *exact* regulile de mai sus, nu o limită numerică fixă din sistem. Strategia de răspuns:

1. Identifică cine deschide ce capăt (citire/scriere) și în ce ordine.
2. Verifică, pentru fiecare `open`, dacă există deja (sau va exista) un proces care deschide capătul complementar — altfel acel `open` se **blochează la nesfârșit** (deadlock de tip "ambii așteaptă unul după altul").
3. Dacă un proces deschide **mai multe** FIFO-uri secvențial, iar fiecare `open` este blocant, trebuie verificat dacă perechea necesară pentru **primul** FIFO poate apărea *fără* ca cealaltă parte să aibă nevoie, la rândul ei, ca tu să-ți fi deschis deja al doilea FIFO (altfel apare un **deadlock circular** între procese, nu o limită de numărare).

Nu există, în general, o limită fixă de "câte FIFO-uri poți deschide" impusă de FIFO-ul însuși (limita reală e numărul de descriptori de fișier permis procesului) — întrebarea vrea, de obicei, să identifice o situație de **blocare reciprocă** sau, dimpotrivă, să confirme că nu există niciun obstacol (răspuns: oricâte, fără limită impusă de natura FIFO).

3.6 De ce nu este indicată comunicarea bidirecțională printr-un singur FIFO

- Un FIFO este, prin construcție, un canal **unidirecțional** (o coadă FIFO de octeți).
- Dacă două procese scriu și citesc din același FIFO, fiecare proces ar putea citi **propriile** mesaje (nu există nicio garanție că un mesaj scris de A va fi citit de B și nu de A însuși), pentru că ordinea de citire/scriere e doar FIFO la nivel de octeți, fără "etichetare" a expeditorului/-destinatarului.
- Soluția corectă: **două** FIFO-uri, unul pentru fiecare direcție ($A \rightarrow B$ și $B \rightarrow A$).

4 Thread-uri: creare și sincronizare

4.1 De ce avem nevoie de thread-uri (recapitulare scurtă)

Un **thread** este o unitate de execuție care partajează spațiul de memorie (variabile globale, heap) cu celelalte thread-uri din același proces — diferit de procese (create prin **fork**), care au spații de memorie separate. Partajarea memoriei este utilă (comunicare directă prin variabile), dar introduce **condiții de cursă** (race conditions) atunci când mai multe thread-uri citesc/scriu aceeași dată fără protecție.

4.2 Creare și sincronizare de bază: `pthread_create` / `pthread_join`

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void* f(void* arg) {
5     printf("salut din thread\n");
6     return NULL;
7 }
8
9 int main() {
10     pthread_t t;
11     pthread_create(&t, NULL, f, NULL); // porneste un thread nou, care
        ruleaza f()
12     pthread_join(t, NULL);           // asteapta terminarea thread-ului
        t
13     return 0;
14 }
```

`pthread_create(&t, attr, func, arg)` creează un thread nou care începe să execute `func(arg)` **concurrent** cu thread-ul apelant (nu se așteaptă să termine).

`pthread_join(t, &ret)` blochează thread-ul apelant până când thread-ul `t` își termină execuția; opțional preia valoarea de retur a funcției.

4.3 Secțiune critică: exemplul clasic de race condition

```
1 int n = 0;
2
3 void* f(void* p) {
4     n++;           // NESIGUR daca mai multe thread-uri executa simultan
5     return NULL;
6 }
```

Instrucțiunea `n++` **nu este atomică**: la nivel de procesor se traduce, de regulă, în trei pași — citește `n`, adună 1, scrie înapoi. Dacă două thread-uri execută aceste trei pași intercalat, una dintre incrementări se poate "pierde" (ambele citesc aceeași valoare veche, ambele scriu aceeași valoare nouă).

Notă de examen: Capcană de examen: dacă enunțul creează **un singur** thread și apoi face `pthread_join` înainte ca `main` să mai toace variabila, **nu există** de fapt o cursă (nu există concurență reală — `main` așteaptă, deci accesul e secvențial). Verifică *întotdeauna* dacă există cu adevărat **acces concurrent** înainte de a adăuga protecție — în multe enunțuri varianta "simplă" (un thread) este doar un șablon pentru ce ar trebui făcut dacă ar fi **mai multe** thread-uri (vezi exemplul cu 100 de "votanți" de mai jos, unde concurența e reală).

5 Mutex (exclusivitate reciprocă)

5.1 Ce este un mutex

Un **mutex** (mutual exclusion) garantează că **cel mult un** thread poate executa o secțiune critică la un moment dat. Orice alt thread care apelează **lock** pe un mutex deja blocat se **suspendă** (blochează) până când mutex-ul este deblocat de cel care îl deține.

5.2 API de bază

```
1 #include <pthread.h>
2
3 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // initializare statica
4 // sau: pthread_mutex_init(&m, NULL);
5
6 pthread_mutex_lock(&m);      // P() - asteapta si "ocupa" mutexul
7 // ... sectiune critica ...
8 pthread_mutex_unlock(&m);    // V() - elibereaza mutexul
```

5.3 Exemplu: protejarea unei secțiuni critice cu mai multe thread-uri

```
1 #include <pthread.h>
2
3 int n = 0;
4 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
5
6 void* f(void* v) {
7     pthread_mutex_lock(&m);
8     if (n < 500) {
9         n++;
10    }
11    pthread_mutex_unlock(&m);
12    return NULL;
13 }
14
15 int main() {
16     pthread_t t[10];
17     for (int i = 0; i < 10; i++)
18         pthread_create(&t[i], NULL, f, NULL);
19     for (int i = 0; i < 10; i++)
20         pthread_join(t[i], NULL);
21     return 0;
22 }
```

Regula generală de protejare a unei secțiuni critice: se identifică citirile/scrierile pe o variabilă partajată care pot fi executate concurrent de mai multe thread-uri, iar **toate** accesele (atât cele de citire, cât și cele de scriere) la acea variabilă se încadrează între **lock** și **unlock** pe **același** mutex.

5.4 Cum prevenim deadlock-urile cauzate de mutex-uri

- **Ordine globală de blocare:** dacă un thread are nevoie de mai multe mutex-uri simultan, toate thread-urile trebuie să le blocheze **în aceeași ordine** (de exemplu, mereu mutexul cu indexul mai mic primul) — astfel se elimină blocările circulare (deadlock).
- **Evitarea blocării imbricate inutile:** cât mai puține mutex-uri ținute simultan, cât mai puțin timp posibil.

- **Folosirea unor încercări non-blocante** (`pthread_mutex_trylock`) pentru a renunța și a reîncerca, în loc de a aștepta la nesfârșit.
- **Un singur mutex global** pentru toate resursele partajate, dacă performanța nu e critică (elimină complet posibilitatea de deadlock între mutex-uri multiple, cu prețul scalabilității).

Notă de examen: Exercițiu clasic: o funcție `f(id)` blochează `m[id%2]` apoi `m[(id+1)%2]`. Dacă este apelată simultan de thread-uri cu `id` par și `id` impar, un thread blochează `m[0]` apoi vrea `m[1]`, în timp ce celălalt blochează `m[1]` apoi vrea `m[0]` — **deadlock circular** (fiecare așteaptă mutexul ținut de celălalt). Soluția: impune o ordine fixă de blocare (de exemplu, blochează întotdeauna mutexul cu indicele mai mic primul), indiferent de `id`.

6 Semafoare POSIX (sem_t)

6.1 Ce este un semafor

Un **semafor** este un contor întreg, protejat de kernel, cu două operații atomice:

- **sem_wait** (P, down): **decrementează** contorul; dacă rezultatul ar fi negativ, thread-ul se **blochează** până valoarea devine > 0 .
- **sem_post** (V, up): **incrementează** contorul și, dacă există thread-uri blocate în **sem_wait**, trezește (cel puțin) unul.

Un semafor **binar** are valori posibile doar 0 și 1 și se comportă funcțional ca un mutex (dar fără conceptul de "proprietar" — orice thread poate face **sem_post**, nu doar cel care a făcut **sem_wait**). Un semafor **numărător** (counting) poate avea valoare inițială $N > 1$ și permite până la N thread-uri simultan în secțiunea protejată.

6.2 API de bază

```
1 #include <semaphore.h>
2
3 sem_t s;
4 sem_init(&s, 0, N);    // al doilea param 0 = partajat intre thread-uri (nu
                        // intre procese)
5                        // N = valoarea initiala (numarul de "permise")
6
7 sem_wait(&s);          // P() - cere un permis (blocheaza daca s == 0)
8 // ... sectiune protejata ...
9 sem_post(&s);          // V() - elibereaza un permis
10
11 sem_destroy(&s);
```

6.3 Exemplu rezolvat: limitarea concurenței (cel mult K thread-uri simultan)

Problema "asigurați-vă că cel mult 3 persoane votează simultan" se rezolvă cu un semafor **numărător** inițializat la 3:

```
1 #include <pthread.h>
2 #include <semaphore.h>
3 #include <stdio.h>
4
5 sem_t sem;
6
7 void* person(void* args) {
8     sem_wait(&sem);    // cere un "loc" din cele 3 disponibile
9     printf("voting ...\n");
10    sem_post(&sem);     // elibereaza locul
11    return NULL;
12 }
13
14 int main() {
15     sem_init(&sem, 0, 3); // maximum 3 persoane simultan
16
17     pthread_t t[100];
18     for (int i = 0; i < 100; i++)
19         pthread_create(&t[i], NULL, person, NULL);
20     for (int i = 0; i < 100; i++)
21         pthread_join(t[i], NULL);
```

```

22     sem_destroy(&sem);
23     return 0;
24 }
25

```

Tipar general: pentru "cel mult K thread-uri simultan într-o zonă", se folosește `sem_init(&s, 0, K)`, cu `sem_wait` la intrarea în zonă și `sem_post` la ieșire.

6.4 Producător-consumator cu semafoare (buffer de capacitate N)

Câte semafoare și ce valori inițiale? Sunt necesare **două** semafoare numărătoare plus, opțional, un mutex pentru accesul la indici (dacă există mai mulți producători/consumatori simultani):

- `sem_t` `goale` — inițializat cu N (numărul de locuri **libere** din buffer la start);
- `sem_t` `pline` — inițializat cu 0 (numărul de elemente **disponibile** de consumat, la start niciunul);
- (opțional) `pthread_mutex_t` `m` — protejează accesul concurent la indicii de scriere/citire dacă există mai mult de un producător sau mai mult de un consumator.

```

1  sem_t goale, pline;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  int buffer[N];
4  int in = 0, out = 0;
5
6  void produce(int val) {
7      sem_wait(&goale);                // asteapta un loc liber
8      pthread_mutex_lock(&m);
9      buffer[in] = val;
10     in = (in + 1) % N;
11     pthread_mutex_unlock(&m);
12     sem_post(&pline);                // semnaleaza un element nou
13     disponibil
14 }
15
16 int consume() {
17     sem_wait(&pline);                // asteapta un element disponibil
18     pthread_mutex_lock(&m);
19     int val = buffer[out];
20     out = (out + 1) % N;
21     pthread_mutex_unlock(&m);
22     sem_post(&goale);                // semnaleaza un loc liber nou
23     return val;
24 }

```

Notă de examen: Întrebare frecventă: "ce înseamnă un semafor binar și ce efect are `P()` apelat de mai multe thread-uri concurrent?" Răspuns: doar **primul** thread care apelează `P()` trece (contorul scade de la 1 la 0); toate celelalte se blochează în `P()` până cineva face `V()`; la `V()`, exact unul dintre cele blocate este trezit (ordinea nu e garantată — nu există "corectitudine" FIFO impusă de standard).

Notă de examen: Capcană: "ce se întâmplă dacă înlocuim `pthread_mutex_lock` cu `sem_post`?" — `sem_post` **nu blochează niciodată** și nu cere niciun "permis"; el doar incrementează contorul. Înlocuirea elimină complet exclusivitatea reciprocă: orice thread poate intra liber în secțiunea critică, oricât de des, **fără nicio restricție** — protecția este complet anulată (nu mai există echivalentul unui lock).

6.5 Stare unui thread la `sem_wait`

Când un thread apelează `sem_wait` pe un semafor cu valoarea curentă 0: trece din starea `RUNNING` în starea `BLOCKED` (suspendat), fiind plasat în coada de așteptare a semaforului, până când un alt thread execută `sem_post`, moment în care trece în `READY` (apoi, la rândul său, în `RUNNING` când planificatorul îi alocă procesorul). Dacă valoarea era > 0 la momentul apelului, thread-ul **nu se blochează deloc** — decrementează contorul și continuă execuția imediat.

7 Read-Write Lock (pthread_rwlock_t)

7.1 Diferența față de mutex

Un mutex permite **un singur** thread în secțiunea critică, indiferent de tipul de acces (citire sau scriere). Un **rwlock** face distincție:

- mai multe thread-uri pot ține **simultan** un rdlock (read lock) — **citirile concurente sunt permise**;
- un wrlock (write lock) este **exclusiv** — niciun alt thread nu poate ține în același timp niciun rdlock sau wrlock.

```
1 #include <pthread.h>
2
3 pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;
4
5 pthread_rwlock_rdlock(&rw);    // citire - se poate tine concurent cu alte
    rdlock-uri
6 // ... citire date partajate ...
7 pthread_rwlock_unlock(&rw);
8
9 pthread_rwlock_wrlock(&rw);    // scriere - exclusiv, ca un mutex
10 // ... modificare date partajate ...
11 pthread_rwlock_unlock(&rw);
```

7.2 Efectul înlocuirii mutex-urilor cu rwlock-uri

Notă de examen: Întrebare tip 1: "Ce s-ar întâmpla dacă toate mutex-urile s-ar înlocui cu rwlock-uri și toate apelurile `mutex_lock` s-ar înlocui cu `rwlock_wrlock`?" Răspuns: **niciun efect funcțional** — **wrlock** este la fel de exclusiv ca un mutex obișnuit, deci comportamentul **corect** al programului se păstrează identic (eventual cu un mic cost suplimentar de performanță datorat structurii mai complexe a `rwlock`-ului). Nu se obține niciun beneficiu de concurență, pentru că nu se folosește deloc `rdlock`.

Notă de examen: Întrebare tip 2 (capcana opusă): "Ce s-ar întâmpla dacă `mutex_lock` s-ar înlocui cu `rwlock_rdlock`?" Răspuns: aici comportamentul **se schimbă** — **rdlock** permite **mai multe thread-uri simultan** în zona care, înainte, era protejată exclusiv. Dacă secțiunea respectivă face și **scrieri** pe date partajate (nu doar citiri), apare din nou o **condiție de cursă** (mai multe thread-uri pot modifica simultan aceeași variabilă) — protecția este efectiv **spartă**.

Regulă de reținut: `wrlock` $\Leftrightarrow$ exclusivitate (ca mutex); `rdlock` $\Rightarrow$ concurență permisă între cititori, dar **nu** oferă protecție împotriva scrierilor concurente nesincronizate.

8 Variabile de condiție (pthread_cond_t)

8.1 De ce avem nevoie de ele

Un mutex protejează accesul la o variabilă partajată, dar nu oferă un mecanism eficient de "**așteaptă până când condiția X devine adevărată**" — fără variabile de condiție, ar trebui să faci *busy-waiting* (verificare repetată în buclă, cu lock/unlock), ceea ce consumă inutil procesorul.

O **variabilă de condiție** permite unui thread să se "culce" eficient (fără să consume CPU) și să fie trezit exact când un alt thread semnalează o schimbare relevantă — **întotdeauna** folosită împreună cu un mutex.

8.2 API de bază

```
1 #include <pthread.h>
2
3 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
4 pthread_cond_t c = PTHREAD_COND_INITIALIZER;
5
6 // Thread care asteapta o conditie:
7 pthread_mutex_lock(&m);
8 while (!conditie_indeplinita) {
9     pthread_cond_wait(&c, &m);
10 }
11 // ... conditia e adevarata, continua in siguranta ...
12 pthread_mutex_unlock(&m);
13
14 // Thread care schimba conditia si semnalizeaza:
15 pthread_mutex_lock(&m);
16 conditie_indeplinita = 1;
17 pthread_cond_signal(&c); // trezeste UN thread care asteapta (sau
    broadcast pentru toate)
18 pthread_mutex_unlock(&m);
```

8.3 Ce face pthread_cond_wait cu mutex-ul primit ca argument

Regulă esențială (foarte des întrebată): `pthread_cond_wait(&c, &m)` execută **atomic** următoarea secvență:

1. **deblochează** mutex-ul `m` (pentru ca alte thread-uri să poată intra în secțiunea critică și să modifice condiția);
2. suspendă thread-ul apelant, punându-l în coada de așteptare a variabilei de condiție `c`;
3. când este trezit (prin `signal/broadcast`), **reblochează** mutex-ul `m` **înainte** de a returna din `pthread_cond_wait`.

Faptul că deblocarea mutex-ului și adormirea sunt **atomice** (nu există o "fereastră" în care alt thread ar putea semnaliza fix între cele două) este exact ce previne pierderea unui semnal ("lost wakeup").

8.4 De ce se reverify condiția într-o buclă while după cond_wait

- **Spurious wakeup**: standardul POSIX permite ca `pthread_cond_wait` să se trezească *occasional* fără niciun `signal/broadcast` real (din motive de implementare la nivel de sistem).
- **Condiție deja "consumată"**: chiar dacă semnalul a fost real, între momentul trezirii și momentul în care thread-ul reușește efectiv să re-obțină mutex-ul, un **alt** thread (trezit mai

întâi, sau care a intrat direct) ar putea fi deja "consumat" condiția (de exemplu, ar fi luat deja ultimul element din buffer).

De aceea, regula corectă este **întotdeauna**:

```
1 while (!conditie) {
2     pthread_cond_wait(&c, &m);
3 }
```

și **niciodată** un simplu if.

8.5 Exemplu: producer-consumer cu variabilă de condiție (alternativă la semafoare)

```
1 #define N 10
2 int buffer[N];
3 int count = 0, in = 0, out = 0;
4
5 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
6 pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
7 pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
8
9 void produce(int val) {
10     pthread_mutex_lock(&m);
11     while (count == N)
12         pthread_cond_wait(&not_full, &m);
13     buffer[in] = val;
14     in = (in + 1) % N;
15     count++;
16     pthread_cond_signal(&not_empty);
17     pthread_mutex_unlock(&m);
18 }
19
20 int consume() {
21     pthread_mutex_lock(&m);
22     while (count == 0)
23         pthread_cond_wait(&not_empty, &m);
24     int val = buffer[out];
25     out = (out + 1) % N;
26     count--;
27     pthread_cond_signal(&not_full);
28     pthread_mutex_unlock(&m);
29     return val;
30 }
```

9 Bariere (pthread_barrier_t)

9.1 Semantica unei bariere

O **barieră** inițializată cu valoarea N garantează că **niciun** thread care apelează `pthread_barrier_wait` pe ea nu trece de acel punct **până** nu se adună exact N thread-uri la barieră; în acel moment, **toate** cele N thread-uri sunt deblocate simultan. O barieră este **reutilizabilă**: după ce se "descarcă" (eliberează N thread-uri), poate aduna din nou alte N apeluri ("ture" succesive).

```
1 #include <pthread.h>
2
3 pthread_barrier_t b;
4 pthread_barrier_init(&b, NULL, N);    // N = numarul de thread-uri necesare
    // pentru eliberare
5
6 void* f(void* a) {
7     // ... faza 1 de lucru ...
8     pthread_barrier_wait(&b);          // asteapta pana ajung N thread-uri
    // aici
9     // ... faza 2 de lucru, garantat dupa ce TOATE thread-urile au
    // terminat faza 1 ...
10    return NULL;
11 }
12
13 pthread_barrier_destroy(&b);
```

9.2 Regulă pentru exerciții de tip "găsiți valori pentru $T, N_1, N_2, \dots$ "

Tiparul clasic: se creează T thread-uri pentru fiecare dintre funcțiile $f_1, f_2, \dots, f_k$ (câte una pe fiecare barieră $b_1, \dots, b_k$). Pentru fiecare barieră b_i , numărul **total** de apeluri `pthread_barrier_wait` pe ea este exact T (un apel per thread creat pentru funcția respectivă).

Condiția de terminare corectă: pentru ca **toate** apelurile pe bariera b_i (inițializată cu N_i) să se elibereze (și deci programul să nu rămână blocat), este necesar și suficient ca

$$T \bmod N_i = 0$$

(adică N_i trebuie să fie un **divizor** al lui T) — altfel, "resturile" de thread-uri ($T \bmod N_i$ thread-uri din ultima "tură" incompletă) rămân blocate la nesfârșit, iar `pthread_join` de pe ele nu se mai termină niciodată.

Exemplu de rezolvare (pentru cazul cu 3 bariere b_1, b_2, b_3 și valori T, N_1, N_2, N_3 **distincte** și strict pozitive): se alege T cu mai mulți divizori distincți, de exemplu

$$T = 6, \quad N_1 = 1, \quad N_2 = 2, \quad N_3 = 3.$$

Verificare: $6 \bmod 1 = 0$, $6 \bmod 2 = 0$, $6 \bmod 3 = 0$ — toate condițiile sunt satisfăcute, iar 6, 1, 2, 3 sunt toate distincte și strict pozitive.

9.3 Efectul unei bariere inițializate cu $N = 1$

Dacă $N = 1$, atunci **fiecare** apel individual la `pthread_barrier_wait` se consideră el însuși un grup complet de 1 thread — deci **nu se blochează niciodată**: orice apel returnează imediat. O barieră cu $N = 1$ este, în efect, un **no-op** (nu sincronizează nimic, pentru că "grupul" e format dintr-un singur thread la un moment dat).

10 Tabel comparativ rapid

Mecanism	Permite concurență?	Utilizare tipică
Mutex	Nu (exclusiv, max. 1)	Protejarea unei secțiuni critice simple
Semafor binar	Nu (ca mutex)	Similar mutex, dar fără "proprietar"
Semafor numărător	Da, până la N	Limitare concurență la N ; producer-consumer
Read-write lock (rdlock)	Da, între cititori	Multe citiri concurente, scrieri exclusive
Read-write lock (wrlock)	Nu (exclusiv)	Echivalent funcțional cu mutex
Variabilă de condiție	N/A (mecanism de așteptare)	"Așteaptă până condiția X e adevărată"
Barieră	N/A (sincronizare de fază)	"Toate N thread-uri ajung aici înainte de a continua"

11 Analiza tipurilor de subiecte de examen

Parcurgând arhiva de subiecte furnizată, întrebările pe FIFO/pipe și pe thread-uri se grupează în următoarele **categorii recurente**. Pentru fiecare categorie: ce se cere de fapt, ce trebuie scris/explicat, și un exemplu concret din arhivă.

11.1 Categorii pentru Pipe / FIFO

- P1. Comportament de blocare la citire/scriere pe pipe.** Se cere descrierea exactă a ce face `read/write` în situații-limită: pipe gol, pipe plin, capete închise. *Exemple din arhivă:* "What does `read` do when the pipe contains less data than required, but is not empty?"; "What does `read` do when the pipe is empty?"; "What will the fragment print?" (cu `pipe()` urmat direct de `read` fără scriere — răspuns: `read` se blochează, pentru că pipe-ul e gol și capătul de scriere e încă deschis în procesul curent, deci nu se ajunge să se afișeze nimic / sau, dacă toate capetele de scriere ar fi închise, `read` ar returna 0). **Strategie:** identifică starea exactă a buffer-ului (gol/plin/parțial) și **câte** capete de scriere/citire mai sunt deschise în acel moment precis din program.
- P2. SIGPIPE / EPIPE — scriere fără cititori.** *Exemplu:* "Explicați ce se întâmplă dacă un proces încearcă să scrie într-un pipe plin al cărui capăt de scriere a fost închis de toate celelalte procese" (vezi nota de la secțiunea 2.4 — capcana este să nu confunzi "capăt de scriere închis de alții" cu "capăt de citire închis" — doar al doilea declanșează SIGPIPE). **Strategie:** SIGPIPE/EPIPE apare **exclusiv** când **toate** capetele de **citire** ale pipe-ului respectiv sunt închise; orice altă situație de "plin" duce la blocare normală a `write`, nu la eroare.
- P3. Reguli de open pe FIFO.** *Exemple:* "What does `open` do before returning, when opening a FIFO?"; "A process opens a FIFO for writing and immediately writes 100 bytes — what happens if no reader exists?" **Strategie:** aplică regulile din secțiunea 3.3 (blocare reciprocă fără `O_NONBLOCK`; `ENXIO` la scriere cu `O_NONBLOCK` fără cititor).
- P4. Numărare/limite de FIFO-uri deschise.** *Exemple:* "How many FIFOs can a process open for reading if they are/will be used by others only for writing?"; "...if they are not and will not be used by any other process?" **Strategie:** vezi secțiunea 3.4 — verifică posibilitatea unui deadlock de tip "ambele părți se așteaptă reciproc"; dacă nu există astfel de blocare, răspunsul e "fără limită impusă de FIFO" (limitat doar de numărul maxim de descriptori al procesului).
- P5. Pipe folosit ca mecanism de sincronizare (semafor improvizat).** *Exemplu:* "Describe briefly how you could use a pipe as a semaphore to synchronize two processes." — vezi secțiunea 2.5.
- P6. Comunicare bidirecțională printr-un singur FIFO — de ce se evită.** Vezi secțiunea 3.5.
- P7. `popen/pclose` legate de pipe-uri** (de ex. de ce se folosește `pclose` și nu `fclose`, sau trimiterea unui șir printr-un `popen` în mod scriere) — **notă:** acestea presupun și cunoștințe de `exec`, care sunt **excluse** din acest ghid conform cererii tale; le recomand pentru o sesiune separată dedicată `exec/popen/dup`.

11.2 Categorii pentru Thread-uri

- T1. Protejarea unei secțiuni critice cu mutex.** Se dă un fragment de cod cu o variabilă globală modificată din mai multe thread-uri (sau "potențial" mai multe) și se cere adăugarea protecției corecte. *Exemplu:* "Protejați secțiunea critică din programul de mai jos" (variabila `n`, `if(n<500) n++;`). **Strategie:** declară un `pthread_mutex_t` global, încadrează **toate** accesurile la variabila partajată între `lock/unlock`. **Verifică** mai întâi dacă enunțul implică de fapt concurență reală (un singur thread + `join` imediat = fără cursă reală).

- T2. Limitarea numărului de thread-uri concurente (semafor numărător).** *Exemplu:* "Asigurați-vă că maximum 3 persoane votează deodată" — vezi secțiunea 6.3 (rezolvare completă). **Strategie:** `sem_init(&s,0,K)`; `sem_wait` înainte, `sem_post` după zona limitată.
- T3. Funcționarea internă a `pthread_cond_wait` și a variabilelor de condiție.** *Exemplu:* "Ce face `pthread_cond_wait` cu mutex-ul primit ca argument?"; "De ce se reverify condiția după `cond_wait`?". **Strategie:** vezi secțiunile 8.3 și 8.4 — răspunsurile sunt **definiții exacte**, nu probleme de programare; trebuie reproduse cât mai fidel (deblocare atomică + reblocare la trezire; spurious wakeup + condiție posibil "consumată" de altcineva).
- T4. Efectul înlocuirii unui mecanism de sincronizare cu altul.** *Exemplu:* "Ce s-ar întâmpla dacă toate mutex-urile s-ar înlocui cu `rwlock`-uri și `mutex_lock` cu `rwlock_wrlock`?"; "...cu `rwlock_rdlock`?"; "...cu `sem_post`?". **Strategie:** analizează dacă noul apel (a) **blochează** alte thread-uri concurente și (b) **este exclusiv**. `wrlock` → comportament identic (exclusiv); `rdlock` → permite concurență, posibil rupe corectitudinea; `sem_post` → elimină complet protecția (nu blochează deloc). Vezi secțiunile 7.2 și 6.3.
- T5. Alegerea de valori pentru parametrii unei bariere.** *Exemplu:* exercițiile cu T, N_1, N_2 (sau N_1, N_2, N_3) pentru bariere multiple — vezi secțiunea 9.2 (rezolvare completă, regula $T \bmod N_i = 0$).
- T6. Efectul unei bariere cu $N = 1$.** Vezi secțiunea 9.3.
- T7. Prevenirea deadlock-urilor (general, sau cu exemplu de cod cu 2+ mutex-uri).** *Exemplu:* "Ce puteți face ca programator pentru a preveni deadlock-urile cauzate de blocarea unor mutex-uri?"; fragmentul cu `m[id%2]` și `m[(id+1)%2]`. **Strategie:** vezi secțiunea 5.3 — **ordine globală de blocare** este răspunsul-cheie cel mai frecvent acceptat.
- T8. Semafoare: definiții și mecanică internă.** *Exemplu:* "What is a binary semaphore and what is the effect of its P method when called by multiple threads?"; "How can you increment a POSIX semaphore?" (răspuns: `sem_post`); "What state transition does a process undergo at `sem_wait`?" (vezi secțiunea 6.5).
- T9. Producător-consumator: câte semafoare și ce valori inițiale.** *Exemplu:* "Consider the producer-consumer problem with a buffer of capacity N. How many semaphores...and what initial values?" — vezi secțiunea 6.4 (răspuns complet: 2 semafoare, `goale=N`, `pline=0`, plus opțional un mutex dacă există mai mulți producători/consumatori).
- T10. Trei apeluri de funcții care asigură exclusivitate reciprocă.** *Exemplu:* "Give three function calls that ensure mutual exclusion." — răspuns tipic: `pthread_mutex_lock`, `sem_wait` (pe semafor binar), `pthread_rwlock_wrlock`.
- T11. Câte thread-uri pentru o sarcină dată ("procesarea a un milion de fișiere").** Întrebare de tip "discuție/justificare", nu rezolvare exactă — se discută echilibrul între paralelism (mai multe nuclee CPU folosite) și overhead-ul de creare/comutare a thread-urilor; răspunsul corect e, de regulă, "un număr de thread-uri apropiat de numărul de nuclee CPU disponibile" (nu un milion), motivat de costul creării și al comutării de context.

11.3 Recomandare de strategie generală pentru examen

1. Pentru **orice** întrebare de tip "ce se afișează / ce se întâmplă", scrie **explicit** starea fiecărei resurse (pipe/FIFO/mutex/semafor) **la momentul exact** al apelului problematic — majoritatea greșelilor vin din presupuneri greșite despre stare, nu din necunoașterea regulii.
2. Pentru întrebările de tip "scrieți cod care...", **identifică mai întâi mecanismul potrivit** (mutex pentru exclusivitate simplă; semafor numărător pentru limitare la K ; `rwlock` pentru citiri concurente + scrieri exclusive; barieră pentru sincronizare de fază; variabilă de condiție pentru "așteaptă condiția X").

3. Pentru întrebările de tip "ce s-ar întâmpla dacă înlocuim X cu Y", răspunde mereu pe baza a **două criterii**: (a) noul apel **blochează** sau nu thread-urile concurente, și (b) este **exclusiv** sau permite concurență — restul deducției rezultă automat din aceste două răspunsuri.
4. Pentru întrebările teoretice scurte (definiții), memorează formulările **exacte** din secțiunile 6.5, 8.3, 8.4 — sunt cele mai des reformulate ca întrebări de "1-2 propoziții" la examen.